第 1 章 引言（Introduction）

《Fundamentals of Computer Graphics》第 5 版 · 完整中文翻译 · 第一批

原书 PDF 第 18–29 页；原书正文第 1–12 页。

计算机图形学（computer graphics）这个术语，指的是使用计算机创建和处理图像的一切活动。本书介绍能够用于创建各种图像的算法工具与数学工具。这些图像可以是逼真的视觉特效、用于传达信息的技术插图，或者漂亮的计算机动画。图形可以是二维的，也可以是三维的；图像可以完全由计算机合成，也可以通过处理照片得到。本书主要讨论基础算法和数学，尤其关注用于生成三维物体和场景的合成图像的方法。

真正进行计算机图形学编程，难免需要了解特定的硬件、文件格式，通常还要熟悉一两个图形 API（见 1.3 节）。计算机图形学是一个发展迅速的领域，因此这些具体知识一直在变化。正因为如此，本书尽可能不依赖某一种硬件或 API。我们鼓励读者结合自己的软件与硬件环境，阅读相关文档来补充书中的知识。幸运的是，图形学领域已经形成了相当稳定的术语和概念体系，因此本书讨论的内容应当能够很好地映射到大多数实际环境中。

> 术语：API 是 application programming interface 的缩写，中文为“应用程序编程接口”。

本章将定义一些基础术语，介绍相关的历史背景，并提供与计算机图形学有关的信息来源。

1.1 图形学的主要领域（Graphics Areas）

强行给任何学科分类都可能有风险，但大多数图形学从业者应该都会认可以下几个主要领域：

- 建模（Modeling）——研究如何以计算机可以存储的数学形式，明确描述物体的形状和外观属性。例如，一个咖啡杯可以表示为一组有序的三维点，再配合连接这些点的插值规则，以及描述光线如何与杯子相互作用的反射模型。
- 渲染（Rendering）——这个术语继承自美术领域，研究如何根据三维计算机模型创建具有明暗效果的图像。
- 动画（Animation）——通过一系列图像制造运动的视觉错觉。动画会用到建模和渲染，但额外引入一个关键问题：物体怎样随时间运动。这个问题通常不属于基础建模与渲染的研究范围。

此外，还有许多与计算机图形学有关的领域。它们是否应该被归为图形学的核心领域，见仁见智。本书至少会涉及这些领域，其中包括：

- 用户交互（User Interaction）——研究鼠标、数位板等输入设备与应用程序之间的接口，以及图像形式的用户反馈和其他感官反馈。从历史上看，这个领域与图形学密切相关，主要是因为图形学研究人员很早就有机会使用今天已经非常普遍的输入输出设备。
- 虚拟现实（Virtual Reality）——试图让用户沉浸于一个三维虚拟世界。这通常至少需要立体图像，以及对头部运动作出响应。真正的虚拟现实最好还应提供声音和力反馈。由于它需要先进的三维图形技术和显示技术，因此往往与图形学密切相关。

- 可视化（Visualization）——通过视觉显示，让用户能够理解复杂信息。可视化问题中经常需要解决图形学方面的技术问题。
- 图像处理（Image Processing）——研究二维图像的处理，在图形学和计算机视觉领域都会用到。
- 三维扫描（Three-Dimensional Scanning）——利用测距技术创建从真实世界测量得到的三维模型。这些模型有助于生成丰富的视觉图像，而处理它们往往需要图形学算法。
- 计算摄影（Computational Photography）——综合运用计算机图形学、计算机视觉与图像处理方法，创造拍摄物体、场景和环境的新方式。

1.2 主要应用（Major Applications）

几乎任何行业都可以利用计算机图形学，不过以下行业是图形学技术的主要使用者：

- 电子游戏（Video Games）——越来越多地使用复杂的三维模型和渲染算法。
- 卡通制作（Cartoons）——经常直接从三维模型渲染图像。许多传统二维卡通使用三维模型渲染的背景，这让镜头视角能够连续移动，而不必耗费大量美术师工时。
- 视觉特效（Visual Effects）——几乎会用到各种计算机图形学技术。现代电影几乎都会利用数字合成，将背景与单独拍摄的前景叠加。许多电影还会通过三维建模和动画创建合成环境、物体，甚至人物，而大多数观众并不会察觉它们并非真实拍摄。
- 动画电影（Animated Films）——使用的许多技术与视觉特效相同，不过不一定追求看起来真实的画面。
- CAD/CAM——分别是计算机辅助设计（Computer-Aided Design）和计算机辅助制造（Computer-Aided Manufacturing）。这些领域使用计算机设计零件与产品，再通过虚拟设计结果指导制造过程。例如，许多机械零件先在三维建模软件中设计，随后由计算机控制的铣床自动加工。
- 仿真（Simulation）——可以理解为对准确性要求更高的电子游戏。例如，飞行模拟器使用复杂的三维图形来模拟驾驶飞机的体验。对于驾驶等安全关键领域的初期训练，以及实际搭建成本过高或过于危险的场景（例如特定消防情境），这类仿真在培训有经验的人员时也很有价值。
- 医学成像（Medical Imaging）——根据扫描得到的患者数据生成有意义的图像。例如，计算机断层扫描（CT）数据集由一个大型三维矩形密度值数组组成。计算机图形学可以把这些数据生成具有明暗效果的图像，帮助医生提取其中最重要的信息。
- 信息可视化（Information Visualization）——为那些并没有“自然外观”的数据创建图像。例如，十只不同股票的价格随时间变化，并没有一个显而易见的自然图像，但巧妙设计的图表能够帮助人们看出数据中的模式。

1.3 图形 API（Graphics APIs）

使用图形程序库的关键环节之一，就是与图形 API 打交道。应用程序编程接口（API）是一组标准化函数，用于执行一组相关操作。图形 API 则是一组用来执行基础图形操作的函数，例如在屏幕窗口中绘制图像和三维表面。

每个图形程序都需要能够使用两类相关的 API：一类是进行视觉输出的图形 API，另一类是获取用户输入的用户界面 API。图形与用户界面 API 目前主要有两种设计范式。

第一种是集成式方法，以 Java 为代表：图形工具包和用户界面工具包被集成为一套可移植的程序包，作为语言的一部分得到完整标准化与支持。

第二种以 Direct3D 和 OpenGL 为代表：绘图命令属于与 C++ 等语言配合使用的软件库，而用户界面软件则是一个独立的部分，可能因操作系统而异。采用后一种方法时，编写可移植代码会比较困难。不过，对于简单程序，可以使用可移植的库封装那些与具体系统相关的用户界面代码。

无论选择哪种 API，基本的图形绘制调用大体相似，本书中的概念都同样适用。

1.4 图形管线（Graphics Pipeline）

如今的每台桌面计算机都拥有强大的三维图形管线。这是一个特殊的软件与硬件子系统，能够高效地以透视方式绘制三维图元。通常，这些系统针对具有共享顶点的三维三角形进行优化。

图形管线的基本操作，是把三维顶点的位置映射到二维屏幕位置，并为三角形计算明暗，使它们既具有真实感，又能按照正确的前后遮挡关系显示。

过去，在有效的前后顺序下绘制三角形，曾经是计算机图形学中最重要的研究问题。不过，现在它几乎总是通过 z-buffer（深度缓冲区）解决。z-buffer 使用一块专门的内存缓冲区，以一种相对直接、近乎蛮力的方法处理遮挡。

事实证明，图形管线中的几何变换几乎完全可以在一个四维坐标空间中完成。这个空间由三个传统的几何坐标，以及第四个有助于透视观察的齐次坐标组成。四维坐标通过 4×4 矩阵和四维向量进行处理。因此，图形管线包含大量用于高效计算、组合这些矩阵和向量的机制。

这个四维坐标系统是计算机科学中最微妙、也最优美的构造之一。它无疑也是学习计算机图形学时需要跨过的最大知识门槛之一。因此，每本图形学教材的前半部分通常都会用相当多的篇幅介绍这些坐标。

图像的生成速度很大程度上取决于需要绘制多少个三角形。因为在许多应用中，交互性比视觉质量更重要，因此值得尽量减少表示模型所需的三角形数量。此外，当从远处观察模型时，与近距离观察相比，所需的三角形数量也会更少。这说明：用可变化的细节层次（level of detail，LOD）来表示模型，会很有价值。

1.5 数值问题（Numerical Issues）

许多图形程序实际上就是三维数值计算程序。因此，数值计算方面的问题往往至关重要。在“过去”，要可靠且可移植地处理这些问题非常困难，因为不同计算机对数字采用不同的内部表示方法，更糟糕的是，它们处理异常情况的方式也不同，而且彼此不兼容。

幸运的是，几乎所有现代计算机都遵循 IEEE 浮点数标准（IEEE Standards Association，1985）。程序员因此可以方便地假设某些数值情形会按照统一规则处理。

IEEE 浮点标准有许多对数值算法编程很有价值的特性，但在图形学中的多数情况下，只需要掌握其中少数几个。首先，也是最重要的，是理解 IEEE 浮点数为实数运算提供了三个“特殊值”：

1. 正无穷（+∞）。它是一个有效的特殊值，比所有有限的有效数值都大。
2. 负无穷（−∞）。它也是一个有效的特殊值，比所有有限的有效数值都小。
3. 非数值（NaN，Not a Number）。它是一个表示无效数值结果的特殊值，可能来自结果未定义的运算，例如 0 除以 0。

IEEE 浮点标准的设计者作出了一些对程序员极为方便的决定。其中很多决定涉及上述三个特殊值如何处理除零等异常情形。此类运算可能记录异常，但程序员在许多情况下可以忽略这些异常。

具体来说，设 a 是一个正实数，那么涉及除以无穷大值的运算遵循以下规则：

$$+a/(+\infty) = +0; \quad -a/(+\infty) = -0$$

$$+a/(-\infty) = -0; \quad -a/(-\infty) = +0$$

原书页边注：IEEE 浮点数对零有两种表示，一种按正零（+0）处理，另一种按负零（−0）处理。二者之间的区别只有在少数情况下才会产生影响，但遇到这些情况时值得记住它们的存在。

其他涉及无穷大的运算，行为也与直觉大致一致。仍然令 a 为正数：

$$\infty + \infty = +\infty; \quad \infty - \infty = \text{NaN}; \quad \infty \times \infty = \infty$$

$$\infty/\infty = \text{NaN}; \quad \infty/a = \infty; \quad \infty/0 = \infty; \quad 0/0 = \text{NaN}$$

包含无穷大的布尔比较也符合预期：

1. 所有有限的有效数值都小于 +∞。
2. 所有有限的有效数值都大于 −∞。
3. −∞ 小于 +∞。

含有 NaN 的表达式遵循的规则很简单：

1. 任何包含 NaN 的算术表达式都会得到 NaN。
2. 任何涉及 NaN 的布尔比较表达式都为假。

译者辨析：这里按原书措辞翻译。严格而言，IEEE 浮点比较中的“不等于”（!=）遇到 NaN 时会返回 true；原书关于“任何布尔表达式都为假”的说法适用于通常的有序大小比较及相等比较，不应直接理解为涵盖所有布尔运算。

IEEE 浮点数最有用的性质之一，可能就是它处理除零的方式：仍然设 a 为正实数，除以零遵循以下规则：

$$+a/(+0) = +\infty; \quad -a/(+0) = -\infty$$

原书页边注：如果运算中可能出现负零（−0），就需要格外小心。

如果程序员能够利用 IEEE 规则，许多数值计算就可以变得简单得多。例如，考虑下面的表达式：

$$a = 1 / (1/b + 1/c)$$

类似表达式会出现在电阻和透镜的计算中。如果除零会导致程序崩溃（IEEE 浮点标准普及之前的许多系统确实如此），就必须编写两个 if 语句，检查 b 或 c 是否很小、是否为零。而使用 IEEE 浮点运算时，如果 b 或 c 等于零，就会如我们所希望的那样得到 a = 0。

避免特殊检查的另一项常见技巧，是利用 NaN 的布尔比较性质。考虑以下代码：

```
a = f(x)
if (a > 0) then
    do something
```

在这里，函数 f 可能返回 ∞ 或 NaN 这样的“棘手”数值，但 if 条件仍然有明确含义：当 a = NaN 或 $a = -\infty$ 时，条件为假；当 $a = +\infty$ 时，条件为真。只要谨慎决定函数应该返回什么值，if 往往不需要额外的特殊检查，就能作出正确选择。这使程序更短、更健壮，也更高效。

1.6 效率（Efficiency）

不存在能让代码效率提高的魔法规则。高效率来自仔细权衡，而且这些权衡对于不同的计算机体系结构也不相同。不过，在可以预见的未来，有一个不错的经验法则：程序员应该比起计算操作的次数，更加关注内存访问模式。

这恰好与二十年前最好的经验法则相反。发生这种转变，是因为内存速度的提高没能跟上处理器速度的提升。既然这种趋势仍会持续，要优化程序，减少内存访问并让访问保持连贯，就会越来越重要。

要让代码运行得更快，一种合理的办法是按以下顺序行动，并且只在确有需要时执行相应步骤：

1. 首先，以尽可能直接、简单的方式编写代码。需要中间结果时就现场计算，而不是事先把所有结果都存储起来。
2. 使用编译器的优化模式进行编译。
3. 使用现有的性能分析工具，找出关键的性能瓶颈。
4. 检查数据结构，寻找改善局部性（locality）的办法。如果可行，让数据单元的大小与目标体系结构的缓存块或内存页大小相匹配。
5. 如果性能分析发现瓶颈确实位于数值计算部分，就检查编译器生成的汇编代码，寻找遗漏的优化机会，再重写源代码以解决发现的问题。

其中最重要的是第一步。大多数所谓的“优化”，只是让代码更加难懂，却并没有提高运行速度。此外，一开始就花时间优化代码，往往还不如把时间用于修复程序错误或增加功能。

还要小心旧教材提出的建议。有些经典技巧，例如用整数代替实数运算，在现代机器上可能已经无法带来性能优势，因为现代 CPU 通常能以与整数运算相当的速度执行浮点运算。不论遇到什么情况，都应该通过性能分析，确认某项优化对于特定的机器和编译器是否确实有价值。

1.7 图形程序的设计与编码（Designing and Coding Graphics Programs）

图形学编程中，有一些常见策略经常很有用。本节提供若干建议，希望你在实现本书所学方法时能够受益。

1.7.1 类设计（Class Design）

任何图形程序的一个关键组成部分，都是为几何实体（例如向量和矩阵）以及图形实体（例如 RGB 颜色和图像）提供良好的类或操作例程。应当尽可能使这些例程清晰、高效。

一个普遍存在的设计问题是：位置（location）和位移（displacement）是否应该使用不同的类，因为它们可以执行的运算不同。例如，把一个位置乘以二分之一，从几何意义上讲没有明确意义；但把一个位移缩小一半是有意义的（Goldman，1985；DeRose，1989）。

对于这一问题，人们远未达成共识。图形学从业者经常会围绕它展开数小时的激烈讨论。不过，为了给出例子，我们先假设不区分位置与位移。

> 原书页边讨论——P.S.（Peter Shirley）：我非常信奉 KISS（“keep it simple, stupid”，保持简单）原则。从这个角度看，把位置和位移拆成两个类的理由还不够充分，不值得为此引入额外复杂度。

> 原书页边讨论——S.M.（Steve Marschner）：我喜欢把点和向量分成不同的类，因为这样能让代码更容易阅读，还可以让编译器帮助发现某些错误。

这意味着可以先编写以下基本类：

- vector2：二维向量类，存储 x、y 两个分量。最好把它们放入长度为 2 的数组中，以便方便地支持索引运算。还应包括向量加法、向量减法、点积、叉积、标量乘法以及标量除法。
- vector3：三维向量类，设计方式与 vector2 类似。
- hvector：具有四个分量的齐次向量（见第 8 章）。
- rgb：RGB 颜色类，存储三个分量。还应支持 RGB 加法、RGB 减法、RGB 逐分量乘法、标量乘法以及标量除法。
- transform：用于执行变换的 4×4 矩阵。应当提供矩阵乘法，以及分别作用于位置、方向和表面法线向量的成员函数。第 7 章将说明，这三类变换是不同的。
- image：二维 RGB 像素数组，并提供图像输出操作。

此外，还可以考虑是否增加表示区间、标准正交基、坐标标架等内容的类。

> 原书页边注——P.S.：你也可以考虑专门设计一个“单位长度向量”类，但以我的经验，这样做造成的麻烦往往大于带来的好处。

1.7.2 Float 与 Double（Float vs. Double）

根据现代计算机体系结构的特点，减少内存占用、保持内存访问的连贯性，是实现高效率的关键。这意味着使用单精度数据是有利的。然而，为避免数值问题，双精度运算又更有吸引力。

具体应该如何权衡取决于程序，但在定义类时，最好能够为数据类型设定一个默认选择。

原书页边讨论——P.S.：我建议几何计算使用 double，而颜色计算使用 float。对于占用大量内存的数据，例如三角形网格，我建议用 float 存储，但通过成员函数访问数据时，把它们转换成 double。

原书页边讨论——S.M.：我的主张是，所有计算都先使用 float，直到有证据表明某段代码确实需要双精度时，再在相应部分引入 double。

1.7.3 调试图形程序（Debugging Graphics Programs）

如果你向程序员询问，可能会发现一个趋势：随着经验增加，他们使用传统调试器的次数越来越少。部分原因是，对复杂程序来说，调试器比调试简单程序时更难使用。

另一个原因是，最难发现的错误往往是概念性的错误，也就是程序员本身实现了错误的东西。如果只是逐步检查变量值，很容易花费大量时间，却仍然发现不了这类问题。我们发现以下几种调试策略对于图形学尤其有用。

科学方法（The Scientific Method）

图形程序中存在一种与传统调试不同、但经常很有用的方法。它的缺点在于：这种方法与程序员在职业生涯早期被教导“不要这么做”的习惯十分相似，因此你可能会觉得自己有点“不守规矩”：我们先生成一张图像，观察其中出了什么问题，然后针对可能的原因提出假设，并通过实验验证这个假设。

例如，某个光线追踪程序可能出现许多分布得有些随机的暗色像素。这是编写光线追踪器时几乎人人都会遇到的经典“阴影痤疮”（shadow acne）问题。传统调试方法在这里并不特别有帮助。我们需要意识到：用于判定阴影的光线，可能与正在着色的表面自身发生了相交。

我们可能注意到，那些暗斑的颜色恰好是环境光颜色，因此缺失的是直接光照。由于阴影会使直接光照消失，可以提出一个假设：这些点实际上不应该处于阴影中，但程序错误地把它们标记成了阴影。

为了验证这个假设，我们可以关闭阴影检测并重新编译程序。如果暗斑因此消失，就能说明这些点确实被错误地判定成了阴影，接下来便可以继续追查具体原因。

这种方法之所以有时非常有效，关键在于：我们不需要一开始就找出某个错误变量值，也不需要立刻确定自己的概念性错误到底是什么。相反，我们通过实验一步步缩小错误原因的范围。通常只需要进行少量实验就能找到问题所在，而且这种调试过程本身也很有趣。

利用图像输出调试信息（Images as Coded Debugging Output）

在很多情况下，从图形程序中获取调试信息最简单的渠道，就是程序生成的图像本身。

如果你想知道某个针对每个像素执行的计算过程中的变量值，可以临时修改程序，把这个变量值直接复制到输出图像中，并跳过通常需要继续执行的其他计算步骤。

例如，如果你怀疑表面法线有问题，从而导致明暗计算错误，就可以把法线向量直接映射为图像颜色：x 分量对应红色，y 分量对应绿色，z 分量对应蓝色。这样，就能得到一幅用颜色编码的示意图，直接看到计算中实际使用的向量。

又例如，如果你怀疑某个值有时会超出允许范围，可以让程序在出现这种情况的像素位置输出鲜红色。其他常见技巧包括：用明显不同的颜色绘制本不应该看见的表面背面；按物体 ID 给像素着色；或者按照计算每个像素花费的工作量着色。

使用调试器（Using a Debugger）

当然，也有一些情况下，尤其是通过科学方法调试似乎得出了互相矛盾的结果时，直接观察程序究竟发生了什么仍然无可替代。

麻烦在于，图形程序往往会对同一段代码执行非常多次，例如每个像素执行一次，或每个三角形执行一次。如果从程序开始处就一步步单步执行，完全不现实。而且，最难的错误往往只会在复杂输入数据下出现。

一种有用的方法是：给错误“设一个陷阱”。首先，确保程序具有确定性：让它使用单线程执行，并且所有随机数都由固定的种子生成。然后，找出哪个像素或三角形出现了问题，在怀疑有错的代码之前，加入一段只会在这个特定案例下执行的语句。

例如，如果像素 (126, 247) 出现了错误，可以加入：

```
if x = 126 and y = 247 then
    print "blarg!"
```

把断点设置在 print 语句上，就能在目标像素即将计算之前进入调试器。有些调试器提供“条件断点”功能，不需要修改源代码就可以达到相同效果。

> 原书页边注：支持固定随机数种子的专用调试模式很有用。

如果程序崩溃，传统调试器仍然能帮助我们准确找到崩溃位置。随后，应当沿着程序执行路径向前追溯，使用断言（assert）和重新编译，逐步找出程序最先出现错误的地方。新增的断言应该保留在程序中，以便将来捕获你可能又会引入的新错误。

这再次意味着，我们不只是依靠传统的单步执行过程，因为单步执行并不会为程序留下这些有价值的断言。

用数据可视化辅助调试（Data Visualization for Debugging）

有时，很难理解程序到底在做什么，因为在最终出错之前，它会计算大量中间结果。这样的情形类似于一个测量了大量数据的科学实验。解决方法也类似：为自己制作好的图表和示意图，从而理解数据究竟意味着什么。

例如，在光线追踪器中，可以编写代码把光线树可视化，以便看到哪些光线路径对某个像素作出了贡献。在图像重采样程序中，可以绘制图表，标出从输入图像中取样的所有位置。

花时间编写代码，以可视化方式呈现程序的内部状态，在将来优化程序时也会有回报，因为你会因此更好地理解程序的行为。

> 原书页边讨论——S.M.：我喜欢把调试输出格式化，让它们恰好成为一段 MATLAB 或 Gnuplot 脚本，执行后就能生成有帮助的图表。

注释（Notes）

本章的软件工程部分受到《Effective C++》系列（Meyers，1995、1997）、极限编程（Extreme Programming）运动（Beck & Andres，2004），以及《The Practice of Programming》（Kernighan & Pike，1999）的影响。实验式调试的讨论，则来自作者与 Steve Parker 的交流。

每年有许多与计算机图形学相关的学术会议，其中包括 ACM SIGGRAPH 和 SIGGRAPH Asia、Graphics Interface、游戏开发者大会（Game Developers Conference，GDC）、Eurographics、Pacific Graphics、High Performance Graphics、Eurographics Symposium on Rendering，以及 IEEE VisWeek 等。通过在网上搜索会议名称，就可以轻松找到相关资料。

本批翻译说明

本章译文对应原书第五版第 1 章全部正文（原书第 1–12 页）。原书的章、节、小节编号、代码片段、公式和页边作者讨论均已保留并以中文呈现。对原文中容易引起误解的 IEEE NaN 描述，另加了明确标注的译者辨析；它不属于原文作者观点。原书无须嵌入的图版，因此本批 PDF 主要是文字、公式和代码。